Predicting Program Exit Code with LLMs and Programming Language Semantics

Lara Marinov
The University of Texas at Austin

Aditya Thimmaiah
The University of Texas at Austin

Jayanth Srinivasa
Cisco Research

Junyi Jessy Li
The University of Texas at Austin

Milos Gligoric
The University of Texas at Austin







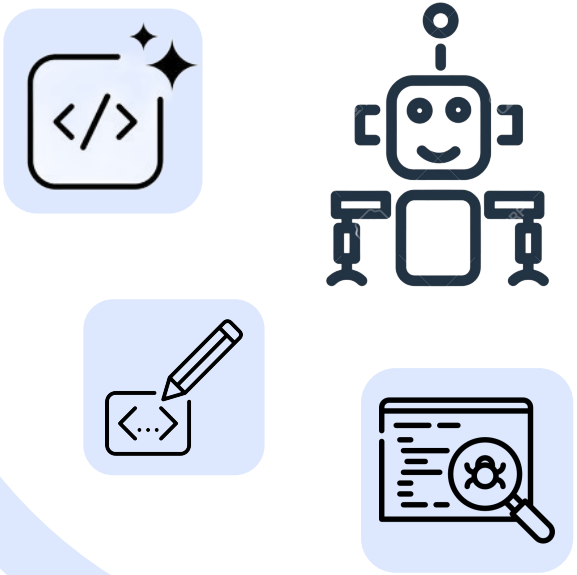
CCF-2217696
CCF2313027
CCF-2403036

Motivation

We ask LLMs to perform a lot of tasks on programs

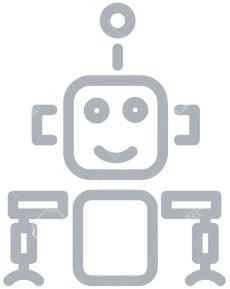
but there are two things we assume/hope:

1. They understand the *meaning* of programs
2. They follow the instructions that we give them



Motivation





We ask LLMs to perform a lot of tasks on programs

but there are two things we assume/hope:

How can we verify that these are both true?

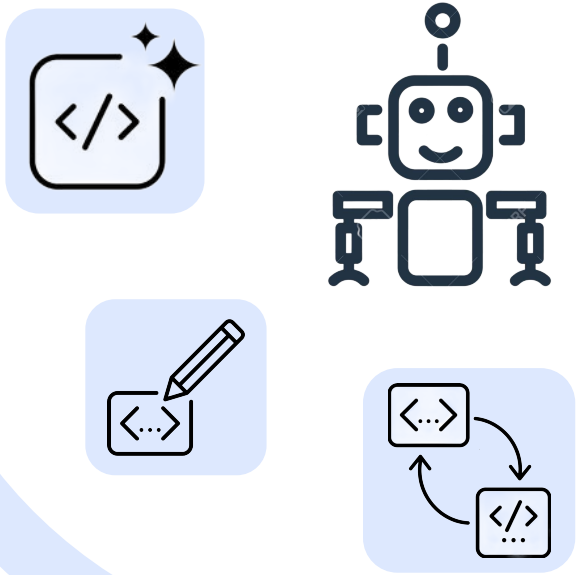
1. They understand the *meaning* of programs
2. They follow the instructions that we give them

A Well Suited Task

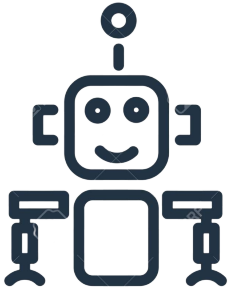
Predicting program execution

Why?

Requires reasoning through the **entire** meaning of the program



Motivation

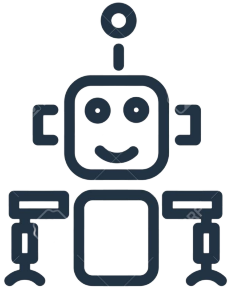


Does this program execute successfully?

```
int a;
int b;
a = 5;
b = (a + 5);
a = (10 % b);
```

✔

Impact of Semantic Change



Does this program execute successfully?

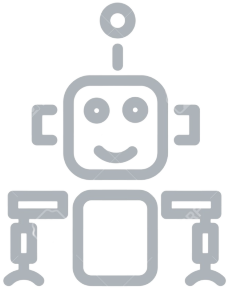
```
int a;
int b;
a = 5;
b = (a + 5);
a = (10 % b);
```

✖

Now, in this language:

"+" ⇨ subtraction

Impact of Semantic Change



Does this program execute successfully?

```
int a;
int b;
a = 5;
b = (a + 5);
a = (10 % b);
```

✖

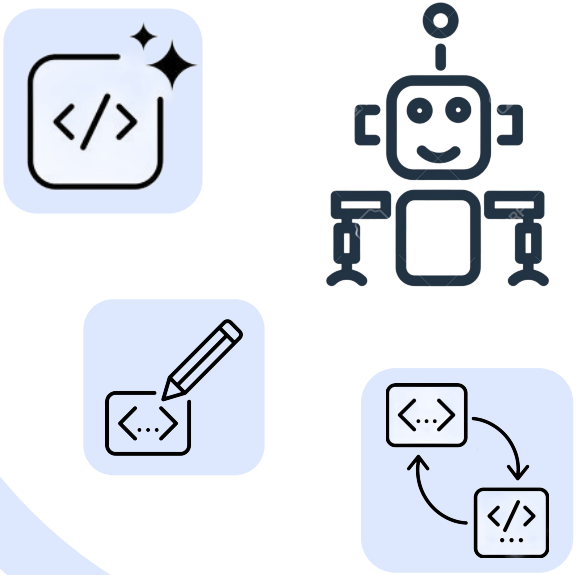
Now, in this language:

"+" ⇨ subtraction

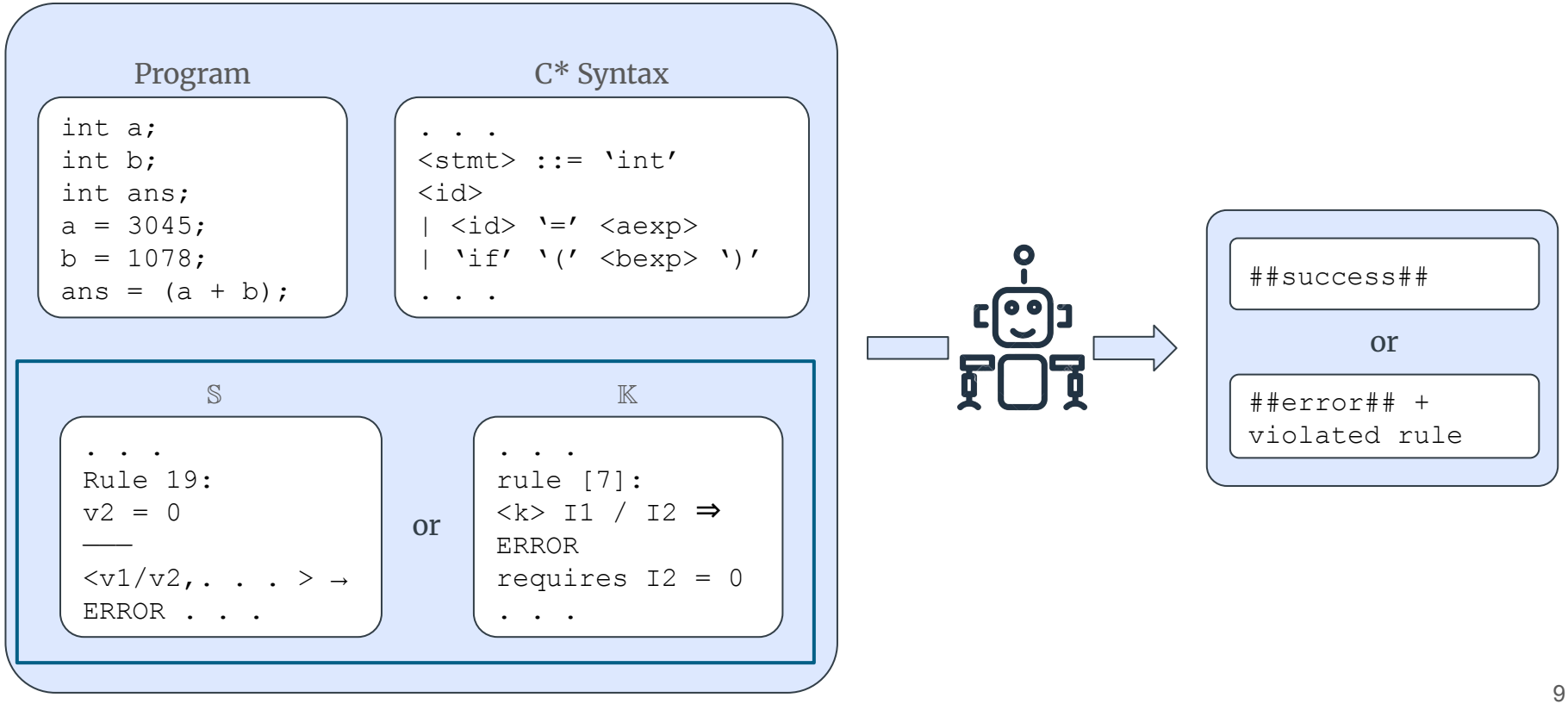
We're changing the ***meaning*** of the symbols, which changes the ***semantics*** of our language

Our Main Question

Do LLMs systematically apply provided formal semantics or do they lean on pre-training priors to predict execution of programs?



The Task: Program Executability Prediction



Semantic Frameworks

𝕊-semantics rules
(Small-Step Operational Semantics)

Rule 18

$$\frac{\texttt{v2} \neq \texttt{0} \qquad \texttt{v3} = \texttt{v1 / v2}}{\langle \texttt{v1 / v2}, \sigma, \chi \rangle \rightarrow \texttt{v3}}$$

Rule 19

$$\frac{\texttt{v2} = \texttt{0}}{\langle \texttt{v1 / v2}, \sigma, \chi \rangle \rightarrow \langle \texttt{ERROR}, \sigma, \chi \rangle}$$

𝕂-semantics rules

Rule 6

<k> I1 {DIV_OP} I2 ⇒ I1 / Int I2 . . . </k>
<rules> L ⇒ L ListItem(“Rule 6”) </rules>
requires I2 ≠ Int 0

Rule 7

<k> I1 {DIV_OP} I2 ⇒ {ERROR} . . . </k>
<rules> L ⇒ L ListItem(“Rule 7”) </rules>
requires I2 = Int 0

Semantic Frameworks

𝕊-semantics rules
(Small-Step Operational Semantics)

𝕂-semantics rules

Rule 18

$$\frac{\texttt{v2} \neq \texttt{0} \qquad \texttt{v3} = \texttt{v1} \ / \ \texttt{v2}}{\langle \texttt{v1} \ / \ \texttt{v2}, \sigma, \chi \rangle \rightarrow \texttt{v3}}$$

Rule 6

<k> I1 {DIV_OP} I2 ⇒ I1 / Int I2 . . . </k>
<rules> L ⇒ L ListItem(“Rule 6”) </rules>
requires I2 ≠ Int 0

Valid Division

When both operands of a `/` expression are values and the right operand value is not zero, the result is obtained by integer division of the left operand value by the right operand value.

Semantic Frameworks

$\mathbb{S}$-semantics rules
(Small-Step Operational Semantics)

Rule 18

$$\frac{\texttt{v2} \neq \texttt{0} \qquad \texttt{v3} = \texttt{v1 / v2}}{\langle \texttt{v1 / v2}, \sigma, \chi \rangle \rightarrow \texttt{v3}}$$

Rule 19

$$\frac{\texttt{v2} = \texttt{0}}{\langle \texttt{v1 / v2}, \sigma, \chi \rangle \rightarrow \langle \texttt{ERROR}, \sigma, \chi \rangle}$$

$\mathbb{K}$-semantics rules

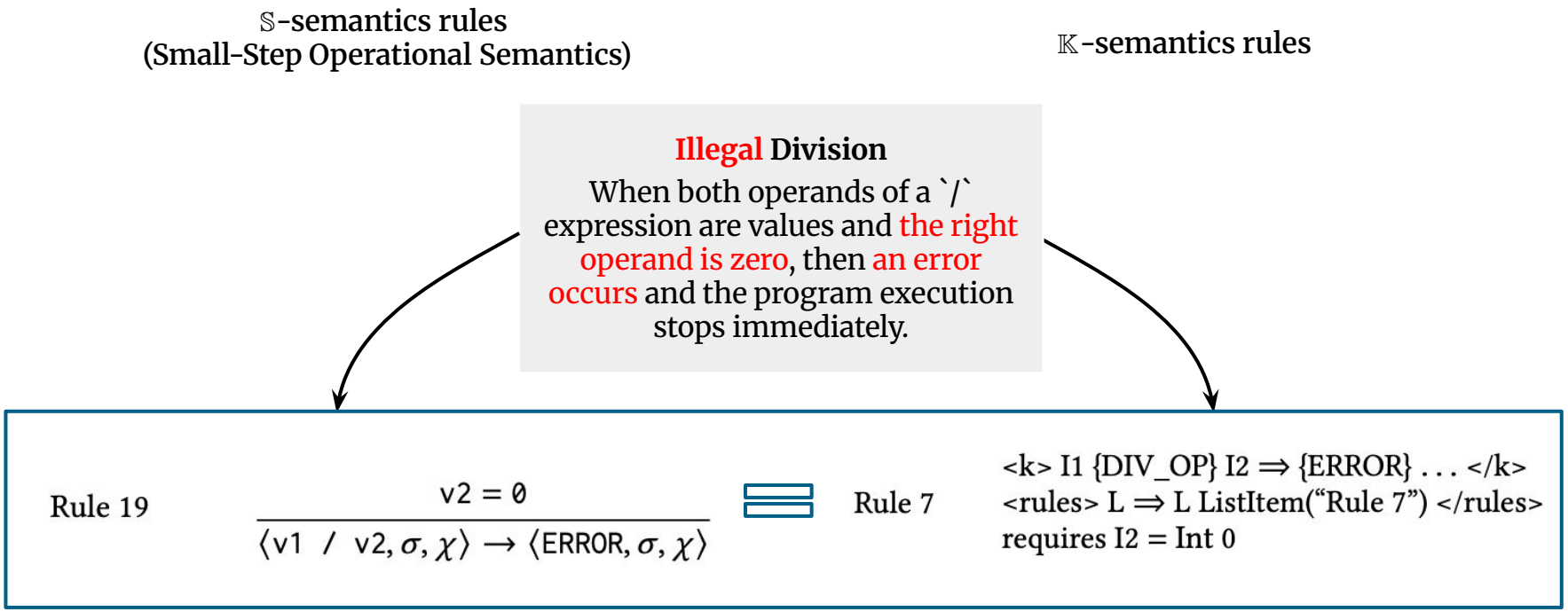
Rule 6

<k> I1 {DIV_OP} I2 ⇒ I1 / Int I2 . . . </k>
<rules> L ⇒ L ListItem(“Rule 6”) </rules>
requires I2 ≠ Int 0

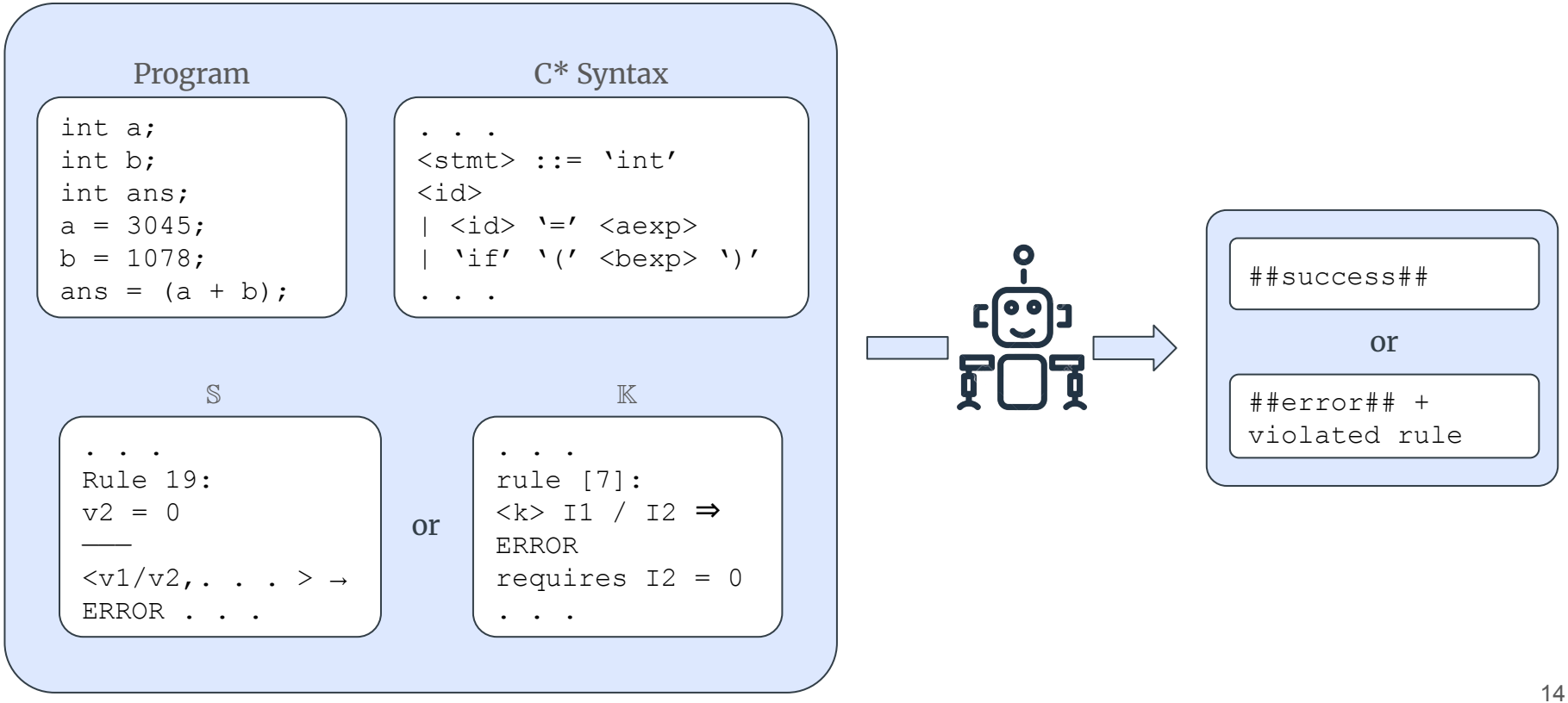
Rule 7

<k> I1 {DIV_OP} I2 ⇒ {ERROR} . . . </k>
<rules> L ⇒ L ListItem(“Rule 7”) </rules>
requires I2 = Int 0

Semantic Frameworks



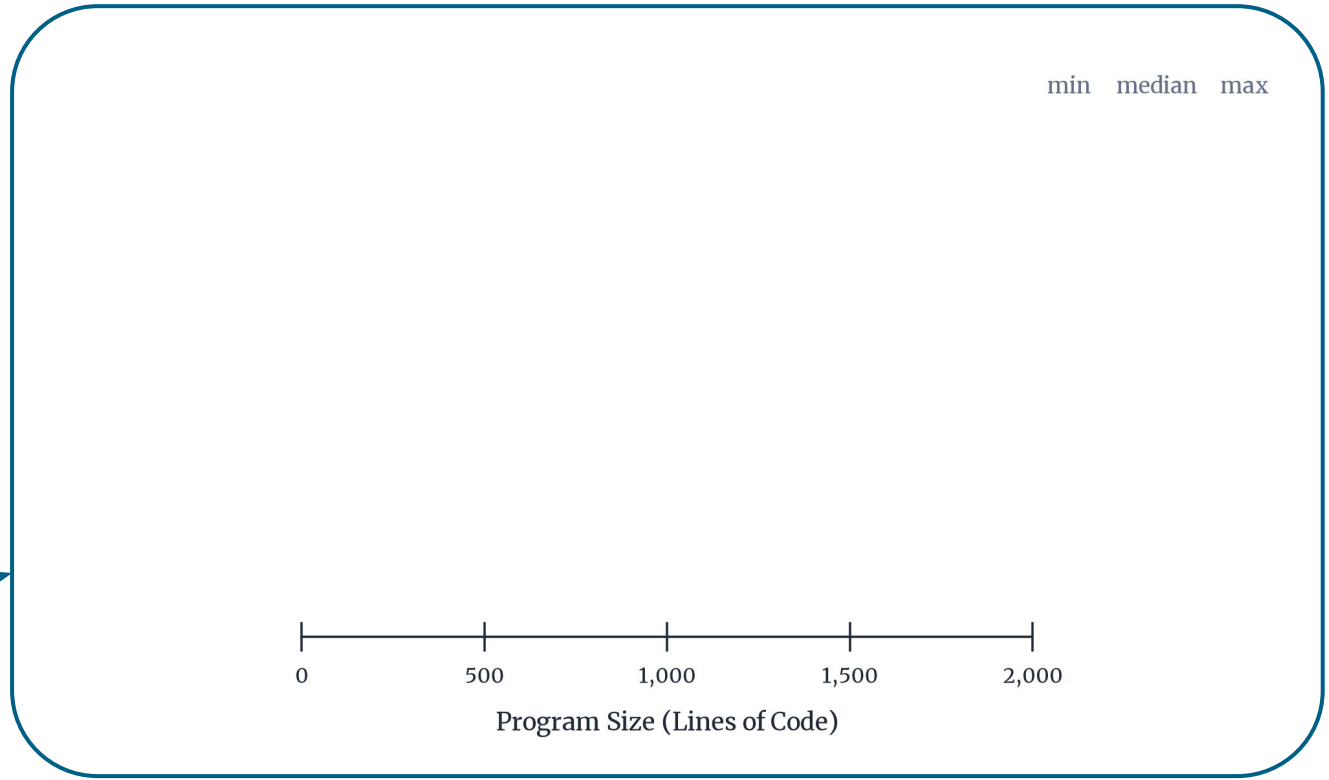
The Task: Program Executability Prediction



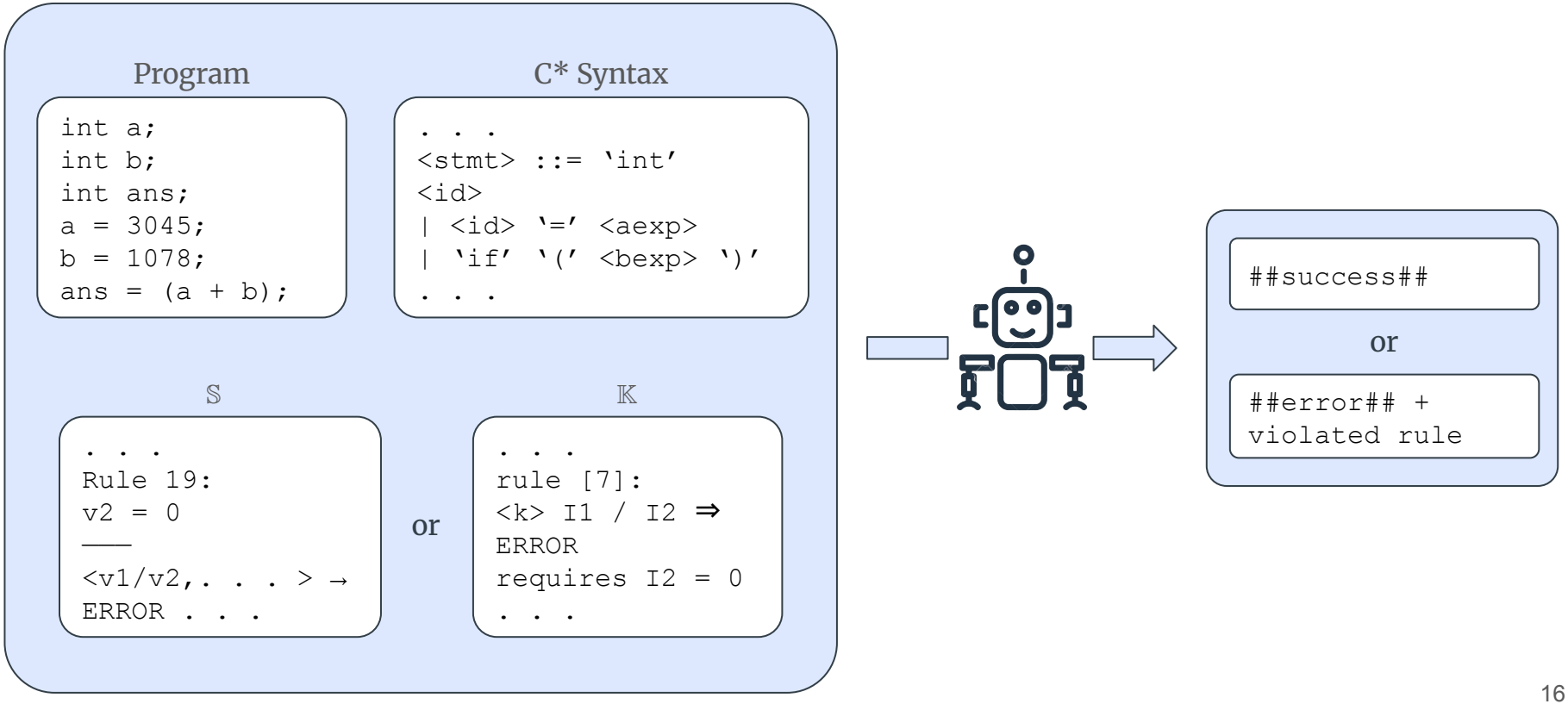
Dataset Splits

Program

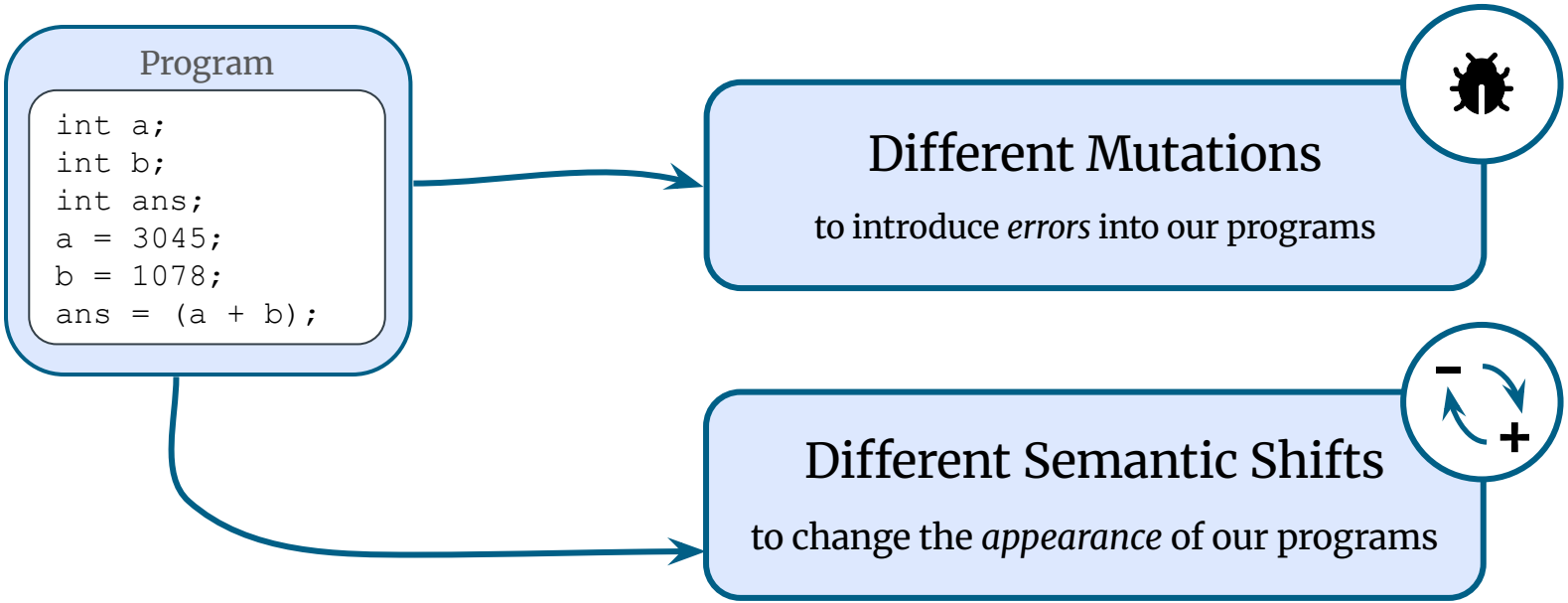
```
int a;
int b;
int ans;
a = 3045;
b = 1078;
ans = (a + b);
```



The Task: Program Executability Prediction



The Task: Program Executability Prediction



Different Mutations



Valid Program

```
int a;
int b;
int ans;
a = 3045;
b = 1078;
ans = (a + b);
```

Break outside loop

```
int a;
int b;
int ans;
a = 3045;
b = 1078;
+ break;
ans = (a + b);
```

Illegal outside of loop

Continue outside loop

```
int a;
int b;
int ans;
+ continue;
a = 3045;
b = 1078;
ans = (a + b);
```

Divide by zero

```
int a;
int b;
int ans;
- a = 3045;
+ a = (3045 / 0);
b = 1078;
ans = (a + b);
```

Illegal operation

Modulo by zero

```
int a;
int b;
int ans;
+ ans = 0;
- a = 3045;
+ a = (3045 % ans);
b = 1078;
ans = (a + b);
```

Variable use before declare

```
int a;
int b;
int ans;
a = 3045;
b = 1078;
- ans = (a + b);
+ ans = (y + b);
```

Illegal use

Different Semantic Shifts

Type	Assignment	Arithmetic				Relational						Logical			Keyword				
Standard	=	+	-	*	/	%	<	<=	>	>=	==	!=	!	&&	||	if-else	while	halt	break	continue

KeywordSwap: models must strictly condition on the provided rules and override pre-trained semantic associations

KeywordObf: ensures models can apply explicitly defined rules in the absence of recognizable patterns from prior exposure

Standard

```
int a;
int b;
int ans;
a = 3045;
b = 1078;
ans = (a + b);
```

KeywordSwap

```
int a;
int b;
int ans;
a = 3045;
b = 1078;
ans = (a - b);
```

KeywordObf

```
int a;
int b;
int ans;
a € 3045;
b € 1078;
ans € (a ★ b);
```

Models

Direct reasoning prompting

Chain-of-Thought (CoT) prompting

Models

Direct reasoning prompting

- Ministral 3 3B
- Ministral 3 8B
- Ministral 3 14B
- Qwen2.5-Coder 3B
- Qwen2.5-Coder 7B
- Qwen2.5-Coder 14B
- Qwen2.5-Coder 32B

Chain-of-Thought (CoT) prompting

- DeepSeek-Qwen 14B
- DeepSeek-Qwen 32B
- Ministral 3 3B-CoT
- Ministral 3 8B-CoT
- Ministral 3 14B-CoT
- Qwen2.5-Coder 3B-CoT
- Qwen2.5-Coder 14B-CoT
- Qwen2.5-Coder 32B-CoT

Performance Across Dataset Splits

Q1: How do models perform as complexity and length of programs increase?

Performance Across Dataset Splits

PrEx accuracy on Human-Written programs.

Models	𝕂-semantics			𝕊-semantics		
	Standard	KeywordSwap (pp)	KeywordObf (pp)	Standard	KeywordSwap (pp)	KeywordObf (pp)
DeepSeek-Qwen 32B	**99**	77 (−22)	**98** (−01)	98	71 (−27)	88 (−10)
Ministral 3 14B-CoT	**99**	79 (−20)	91 (−08)	**99**	**78** (−21)	**89** (−10)
Qwen2.5-Coder 32B-CoT	**99**	**81** (−18)	72 (−27)	97	**78** (−19)	68 (−29)

PrEx accuracy on Fuzzer-Generated programs.

Models	𝕂-semantics			𝕊-semantics		
	Standard	KeywordSwap (pp)	KeywordObf (pp)	Standard	KeywordSwap (pp)	KeywordObf (pp)
DeepSeek-Qwen 32B	**89**	**58** (−31)	**68** (−21)	**90**	**57** (−33)	**57** (−33)
Ministral 3 14B-CoT	88	54 (−34)	66 (−22)	76	51 (−25)	52 (−24)
Qwen2.5-Coder 32B-CoT	73	48 (−25)	24 (−49)	77	44 (−33)	29 (−48)

Models struggle when program complexity increases

Performance on KeywordSwap & KeywordObf

Q2: How do models perform when symbol meanings are swapped and new symbols are introduced?

Performance on KeywordSwap and KeywordObf

PrEx accuracy on Human-Written programs.

Models	𝕂-semantics			𝕊-semantics		
	Standard	KeywordSwap (pp)	KeywordObf (pp)	Standard	KeywordSwap (pp)	KeywordObf (pp)
DeepSeek-Qwen 32B	**99**	77 (−22)	**98** (−01)	98	71 (−27)	88 (−10)
Ministral 3 14B-CoT	**99**	79 (−20)	91 (−08)	**99**	**78** (−21)	**89** (−10)
Qwen2.5-Coder 32B-CoT	**99**	**81** (−18)	72 (−27)	97	**78** (−19)	68 (−29)

PrEx accuracy on Fuzzer-Generated programs.

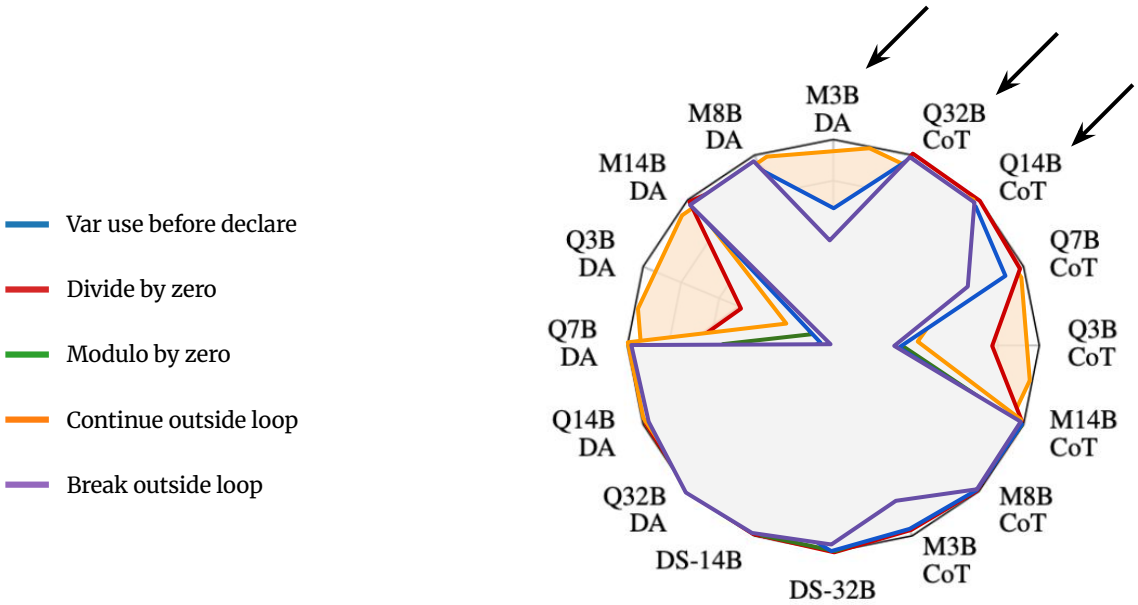
Models	𝕂-semantics			𝕊-semantics		
	Standard	KeywordSwap (pp)	KeywordObf (pp)	Standard	KeywordSwap (pp)	KeywordObf (pp)
DeepSeek-Qwen 32B	**89**	**58** (−31)	**68** (−21)	**90**	**57** (−33)	**57** (−33)
Ministral 3 14B-CoT	88	54 (−34)	66 (−22)	76	51 (−25)	52 (−24)
Qwen2.5-Coder 32B-CoT	73	48 (−25)	24 (−49)	77	44 (−33)	29 (−48)

Models cannot condition on or apply explicitly provided semantic rules

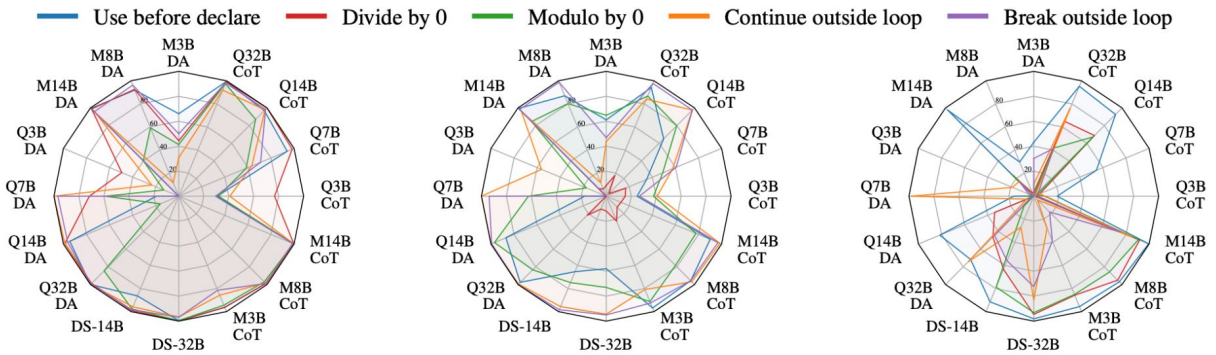
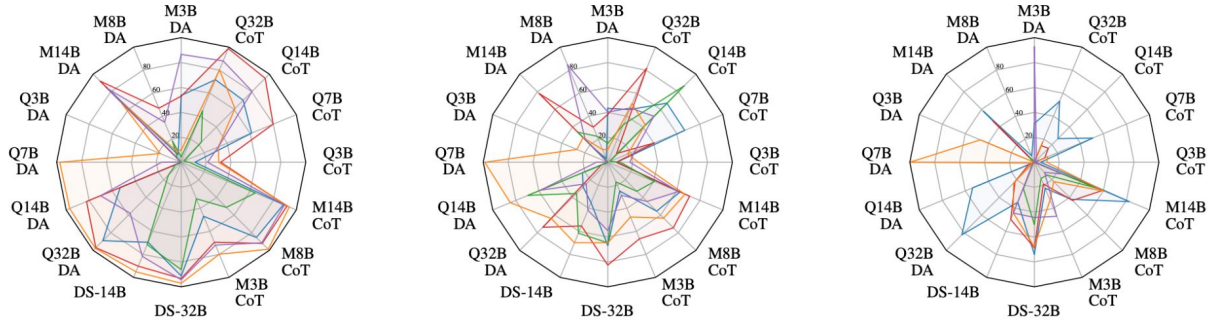
Performance on Rule Identification

Q3: Which errors and rules do models most reliably identify?

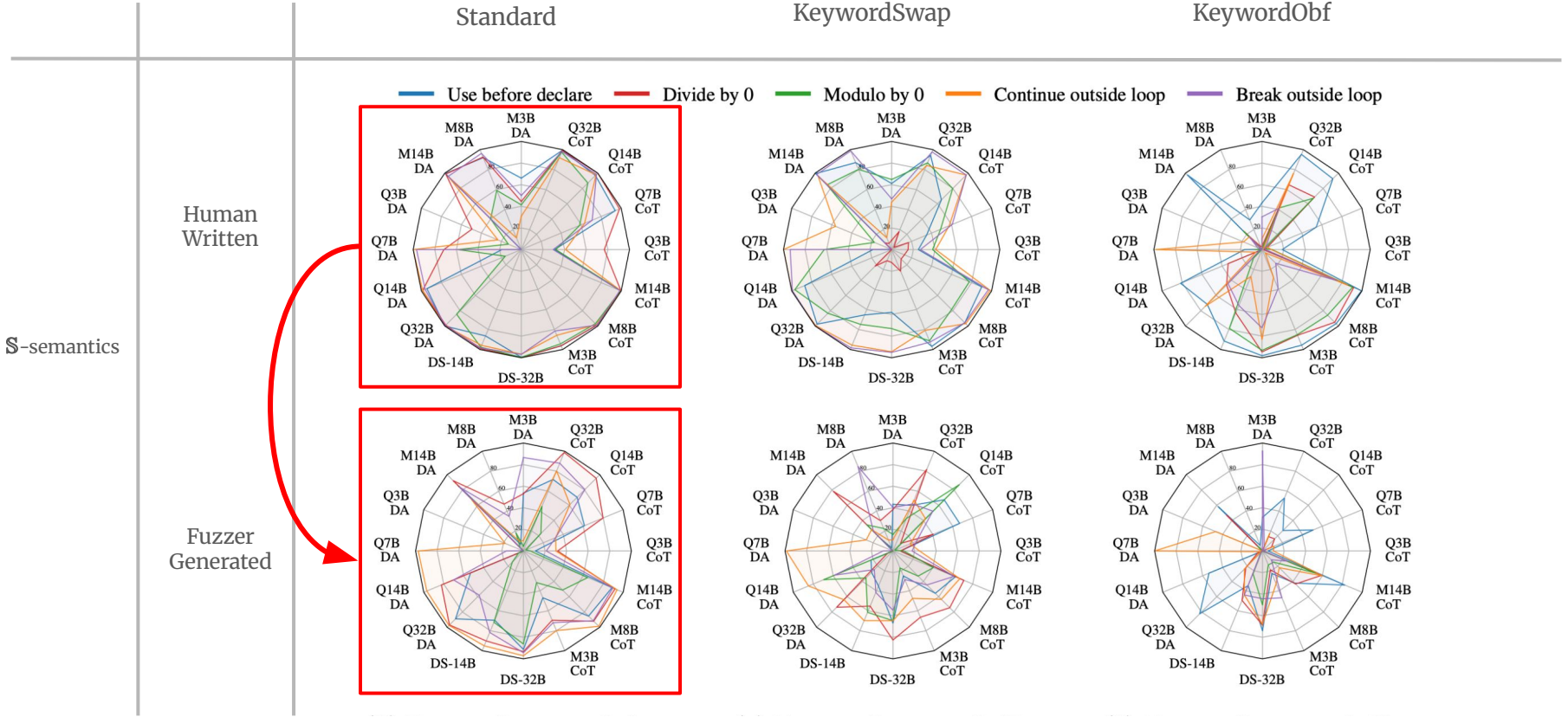
Performance on Rule Identification



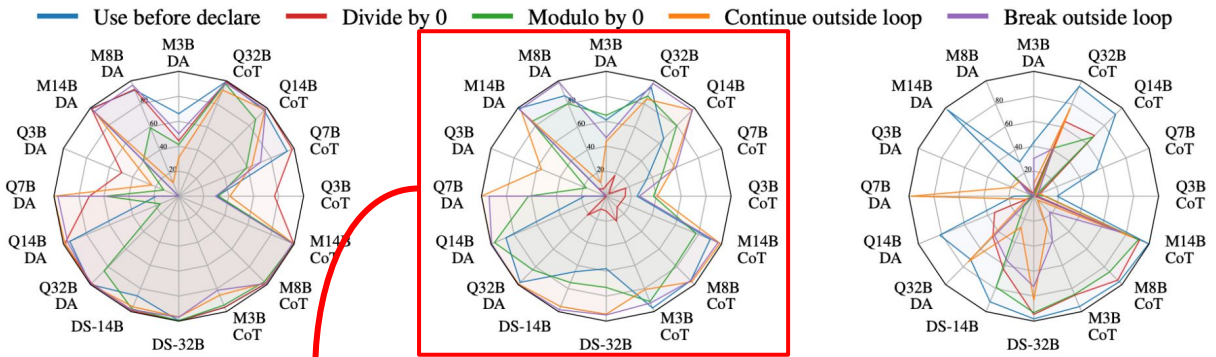
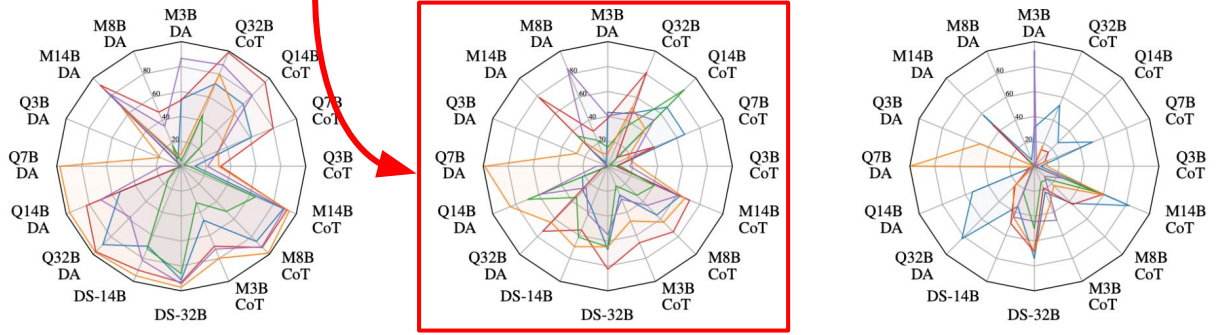
Performance on Rule Identification

		Standard	KeywordSwap	KeywordObf
𝕊-semantics	Human Written			
	Fuzzer Generated			

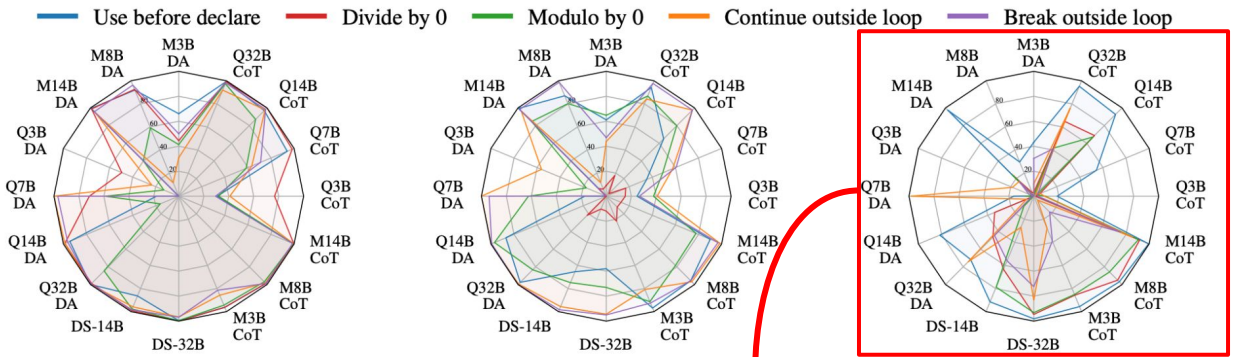
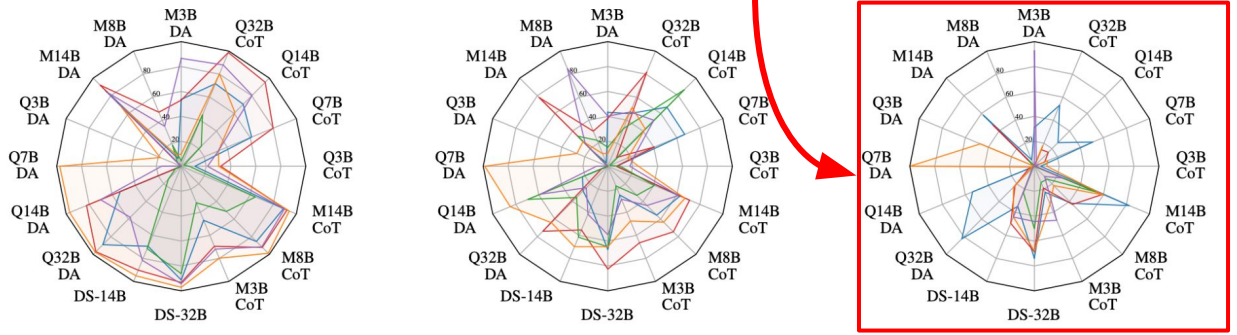
Performance on Rule Identification



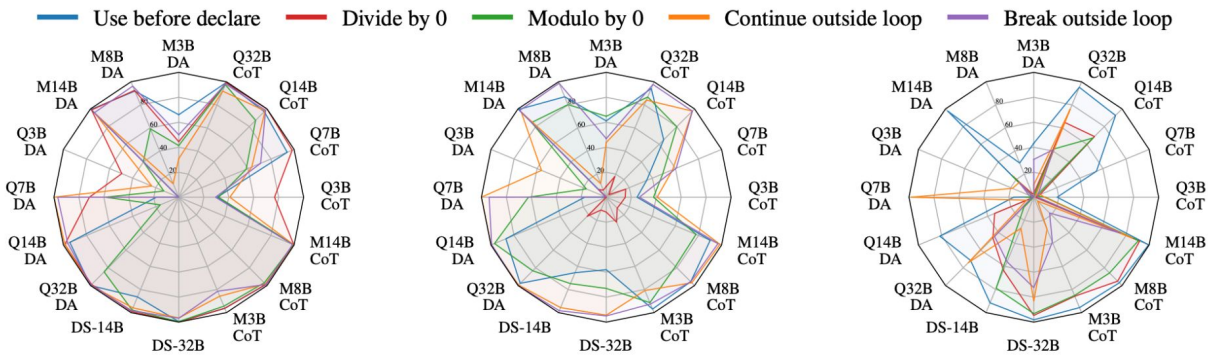
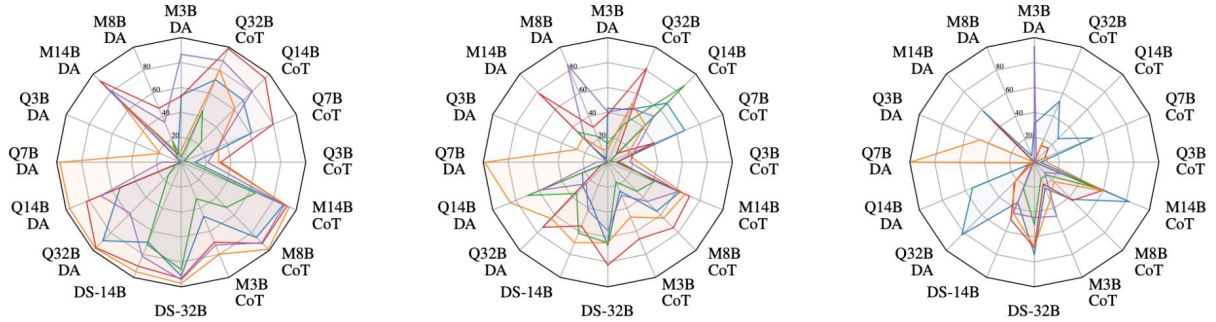
Performance on Rule Identification

		Standard	KeywordSwap	KeywordObf
𝕊-semantics	Human Written			
	Fuzzer Generated			

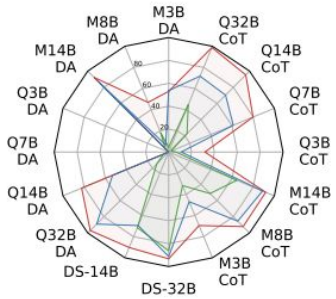
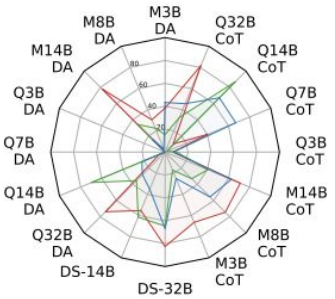
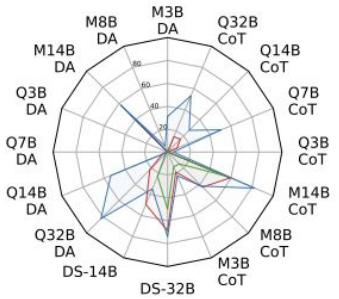
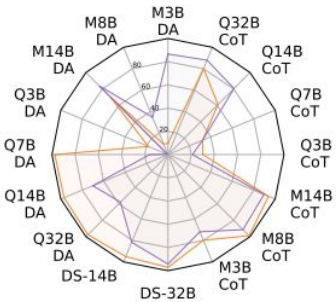
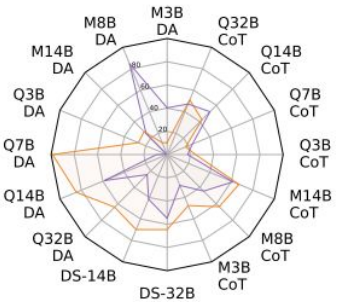
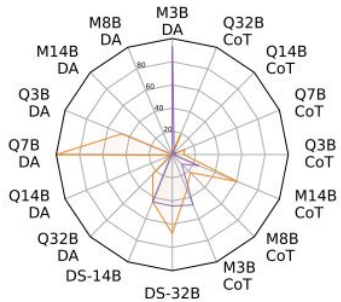
Performance on Rule Identification

		Standard	KeywordSwap	KeywordObf
𝕊-semantics	Human Written			
	Fuzzer Generated			

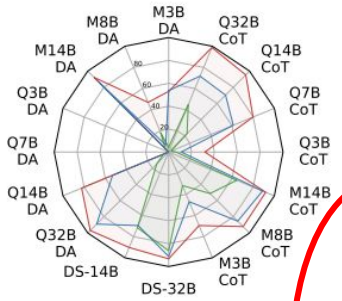
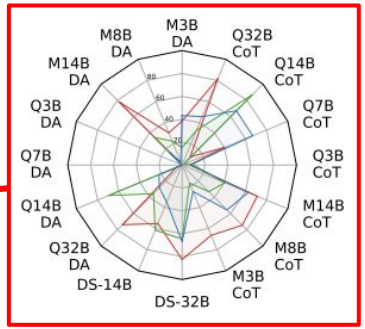
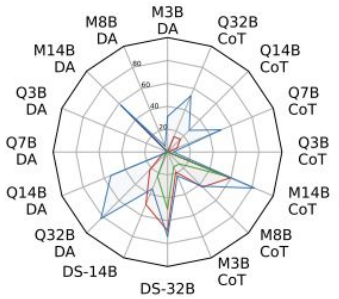
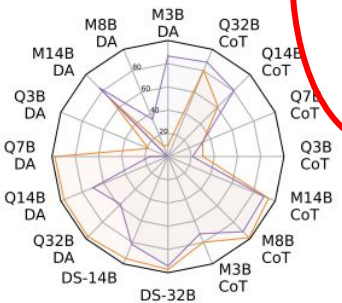
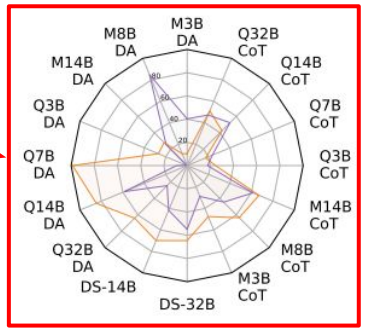
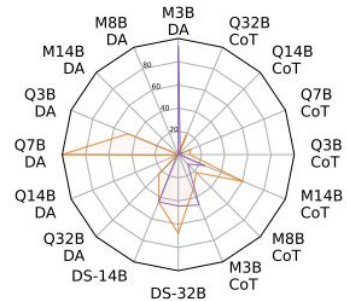
Performance on Rule Identification

$\mathbb{S}$-semantics		Standard	KeywordSwap	KeywordObf
	Human Written			
	Fuzzer Generated			

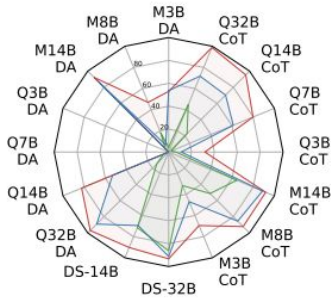
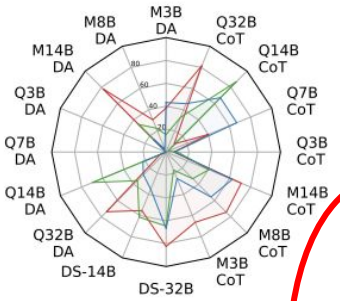
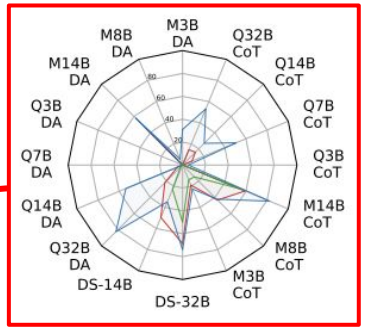
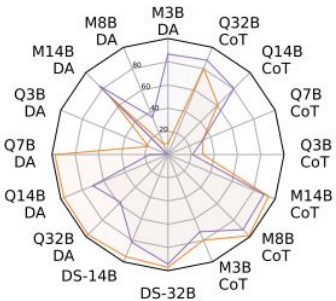
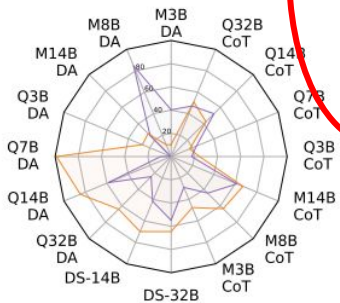
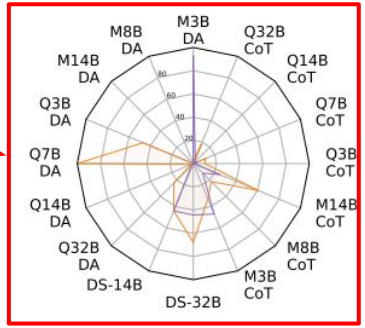
Performance on Rule Identification

			Standard	KeywordSwap	KeywordObf
𝕊-semantics	Fuzzer Generated	{divide, modulo} by zero, var use before declare			
	Fuzzer Generated	{continue, break} outside loop			

Performance on Rule Identification

			Standard	KeywordSwap	KeywordObf
𝕊-semantics	Fuzzer Generated	{divide, modulo} by zero, var use before declare			
	Fuzzer Generated	{continue, break} outside loop			

Performance on Rule Identification

			Standard	KeywordSwap	KeywordObf
𝕊-semantics	Fuzzer Generated	{divide, modulo} by zero, var use before declare			
	Fuzzer Generated	{continue, break} outside loop			

How do models fail?

LLMs Lean on Priors, Not Programming Language Semantics

Aditya Thimmaiah [1] Jiyang Zhang [1] Jayanth Srinivasa [2] Junyi Jessy Li [1] Milos Gligoric [1]

Accepted at ICML'26

Introduce PLSemanticsBench with three, more complicated tasks



Conclusions

- Introduced Program Executability Prediction (PrEx), a task that asks models to predict program execution conditioned on programming language semantics

Conclusions

- Introduced Program Executability Prediction (PrEx), a task that asks models to predict program execution conditioned on programming language semantics
- Investigated whether LLMs apply explicitly provided programming language semantics or fall back on pre-training priors

Conclusions

- Introduced Program Executability Prediction (PrEx), a task that asks models to predict program execution conditioned on programming language semantics
- Investigated whether LLMs apply explicitly provided programming language semantics or fall back on pre-training priors
- Extended PLSemanticsBench dataset with invalid programs produced by five semantics-aware invalid transformations

Conclusions

- Introduced Program Executability Prediction (PrEx), a task that asks models to predict program execution conditioned on programming language semantics
- Investigated whether LLMs apply explicitly provided programming language semantics or fall back on pre-training priors
- Extended PLSemanticsBench dataset with invalid programs produced by five semantics-aware invalid transformations
- Evaluated open-source LLMs under two semantic formalisms, two semantic shifts, and three program data splits

Conclusions

- Introduced Program Executability Prediction (PrEx), a task that asks models to predict program execution conditioned on programming language semantics
- Investigated whether LLMs apply explicitly provided programming language semantics or fall back on pre-training priors
- Extended PLSemanticsBench dataset with invalid programs produced by five semantics-aware invalid transformations
- Evaluated open-source LLMs under two semantic formalisms, two semantic shifts, and three program data splits

LLMs do not yet reliably reason from supplied formal semantics and still heavily rely on patterns learned in their pre-training.

Predicting Program Exit Code with LLMs and Programming Language Semantics

Lara Marinov
The University of Texas at Austin

Aditya Thimmaiah
The University of Texas at Austin

Jayanth Srinivasa
Cisco Research

Junyi Jessy Li
The University of Texas at Austin

Milos Gligoric
The University of Texas at Austin

TEXAS
The University of Texas at Austin

marinov@utexas.edu

CISCO

NSF
CCF-2217696
CCF2313027
CCF-2403036